PART A

Experiment 6

A.1 Aim:Containerizing Applications using Docker: Creating Dockerfiles and Multi-service Deployment with Docker ComposeA.

A.2 Prerequisite.

· Windows 10/11 (64-bit), Ubuntu, or macOS

· Docker Desktop installed

· WSL2 enabled (for Windows)

· Visual Studio Code (recommended)

·Basic command line (CMD/PowerShell or Termina

A.3 Outcome

By the end of this practical, students should be able to:

1.Understand what Docker is.

2.Create a `Dockerfile`.

3.Build a Docker image.

4.Run a container.

A.4 Theory: Why do we need Docker?

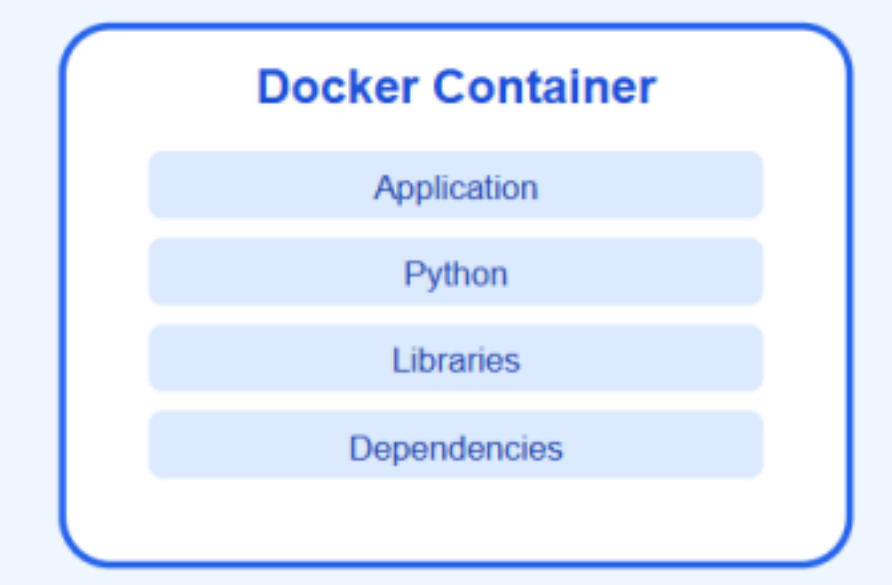
Without Docker

A Python application needs Python, libraries, and dependencies installed separately on every computer.

Problem: Different versions,
Missing libraries,
"It works on my laptop" issue

With Docker

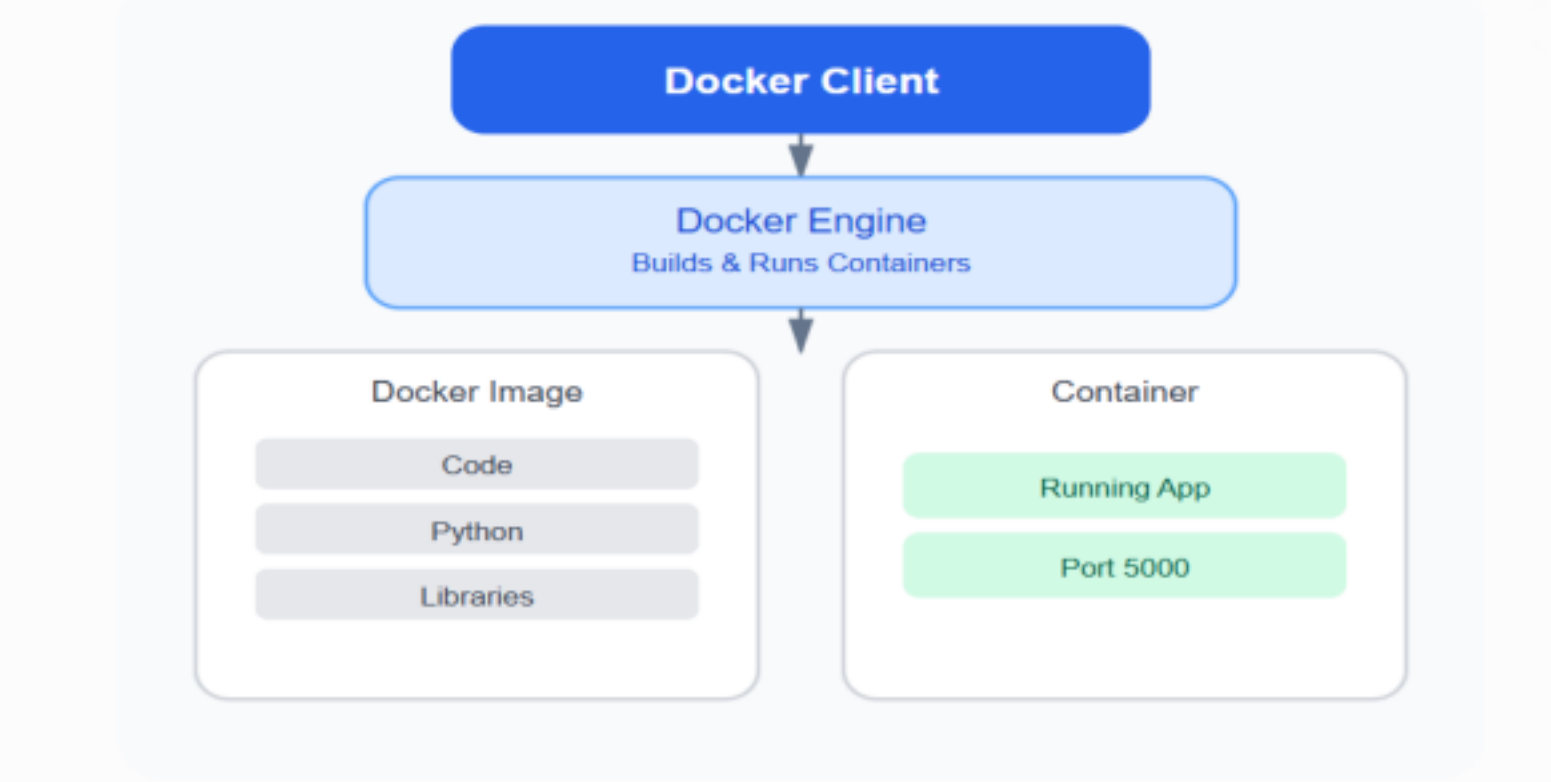
Everything is packed into one container.



Definition (Exam):

> Docker is a containerization platform that packages an application along with its dependencies into a lightweight, portable container.

Docker Architecture



Components

Component	Meaning
Docker Client	Command line (`docker run`)
Docker Engine	Runs Docker

Image Blueprint of application
Container Running instance of image
Docker Hub Online image repository

Memory trick: Image = Recipe

Container = Cooked Food

Step 1: Install Docker

For Windows:

Verify:

```
docker --version
```

Output:

```
Docker version 28.x.x
```

Step 2: Create a Simple Application

Folder structure:

```
MyApp/
 │
 ├── app.py
 ├── requirements.txt
 └── Dockerfile
```

app.py

```
from flask import Flask
app = Flask(__name__)
@app.route("/")def home():
 return "Hello AI&DS Students!"
app.run(host="0.0.0.0", port=5000)
```

requirements.txt

```
flask
```

Step 3: Create Dockerfile (Most Important)

A Dockerfile tells Docker how to build the image.

```
FROM python:3.11
WORKDIR /app
COPY . .
```

```
RUN pip install -r requirements.txt
EXPOSE 5000
CMD ["python", "app.py"]
```

Explain every line

Line	Purpose
`FROM`	Base operating environment
`WORKDIR`	Working folder inside container
`COPY`	Copies project files
`RUN`	Installs libraries
`EXPOSE`	Opens application port
`CMD`	Starts application

Viva: Difference between `RUN` and `CMD`?

- RUN → executes during image creation.
- CMD → executes when container starts.

Step 4: Build Docker Image

Command:

```
docker build -t myapp .
```

Meaning:

- `build` → create image
- `-t` → tag/name
- `.` → current folder

Result:

```
Successfully built
Successfully tagged myapp
```

Check images:

```
docker images
```

Example:

Repository	Tag

myapp latest

Step 5: Run Container

```
docker run -d -p 5000:5000 myapp
```

Explain

Option	Meaning
`-d`	Run in background
`-p`	Map ports
`5000:5000`	PC Port → Container Port

Browser:

http://localhost:5000

Output:

Hello AI&DS Students!

Check running containers:

```
docker ps
```

Step 6: Why Docker Compose?

Suppose you have a Web App + Database.

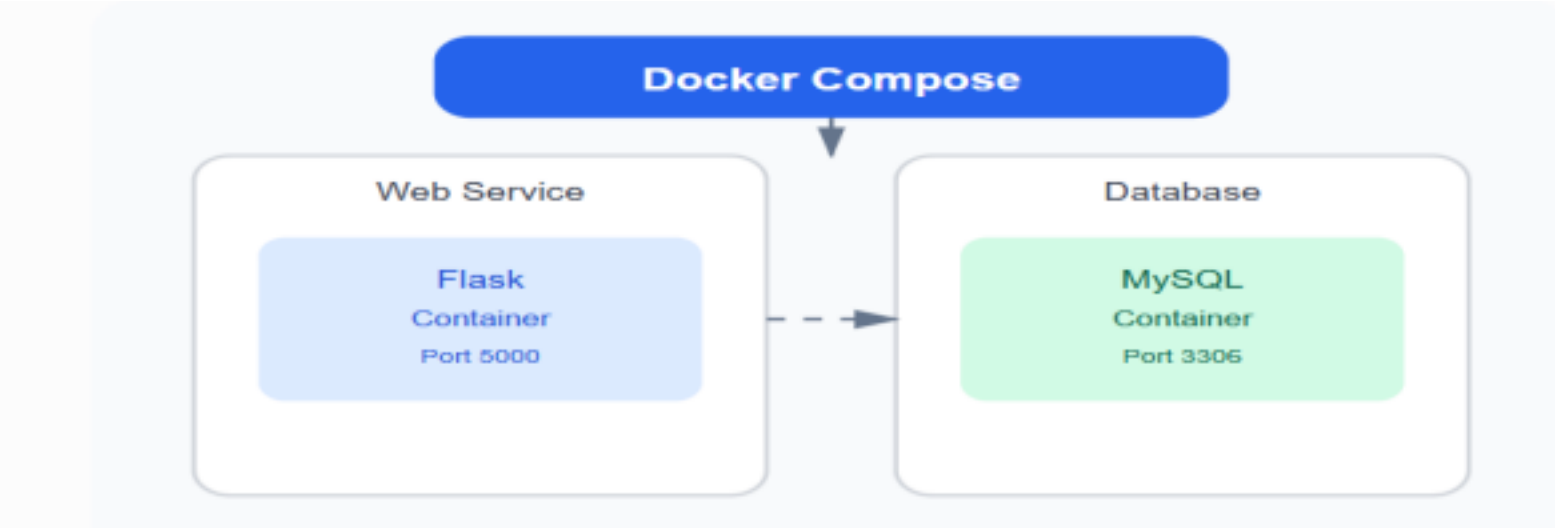
Without Compose:

```
docker run mysql
docker run flask
docker run redis
```

Too many commands!

Docker Compose manages everything using one YAML file. **Step 7:**

Multi-Service Architecture



Two containers communicate automatically.

Step 8: docker-compose.yml

```
version: "3.9"

services:

 web:
 build: .
 ports:
 - "5000:5000"

 database:
 image: mysql:8.0
 environment:
 MYSQL_ROOT_PASSWORD: root123
 MYSQL_DATABASE: college
```

Explain

Keyword	Meaning
`services`	List of containers
`web`	Flask application
`build`	Uses Dockerfile
`ports`	Port mapping
`image`	Download MySQL
`environment`	Password & DB name

Step 9: Start Everything

```
docker compose up
```

Output:

```
Creating network...
Creating web...
Creating database...
```

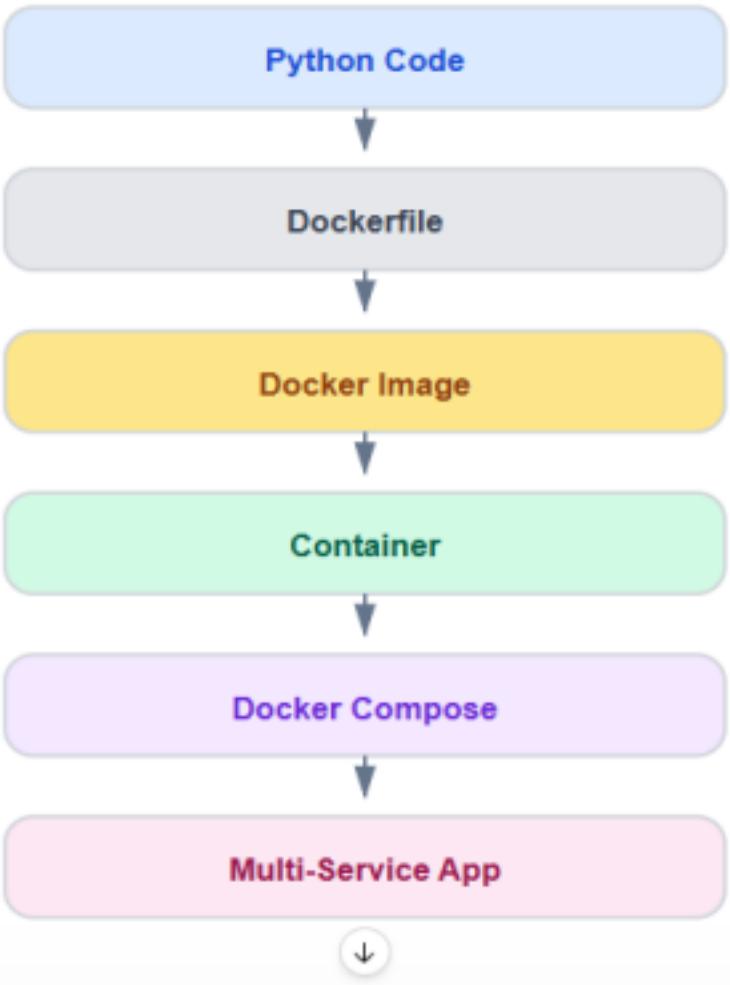
Background mode:

```
docker compose up -d
```

Stop:

```
docker compose down
```

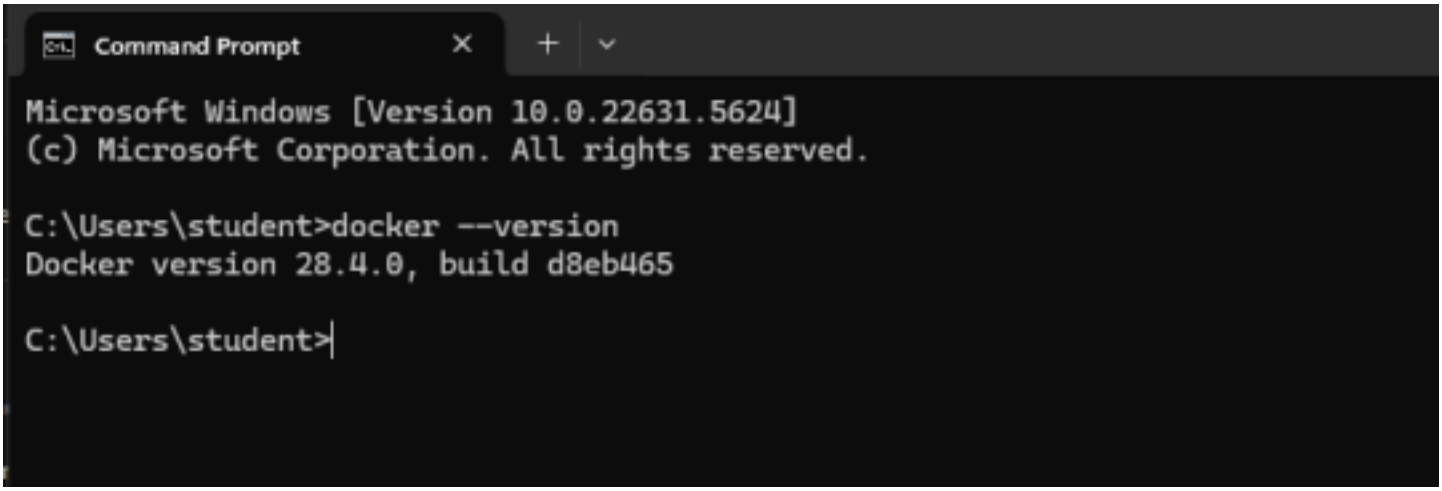
Step 10: Complete Workflow

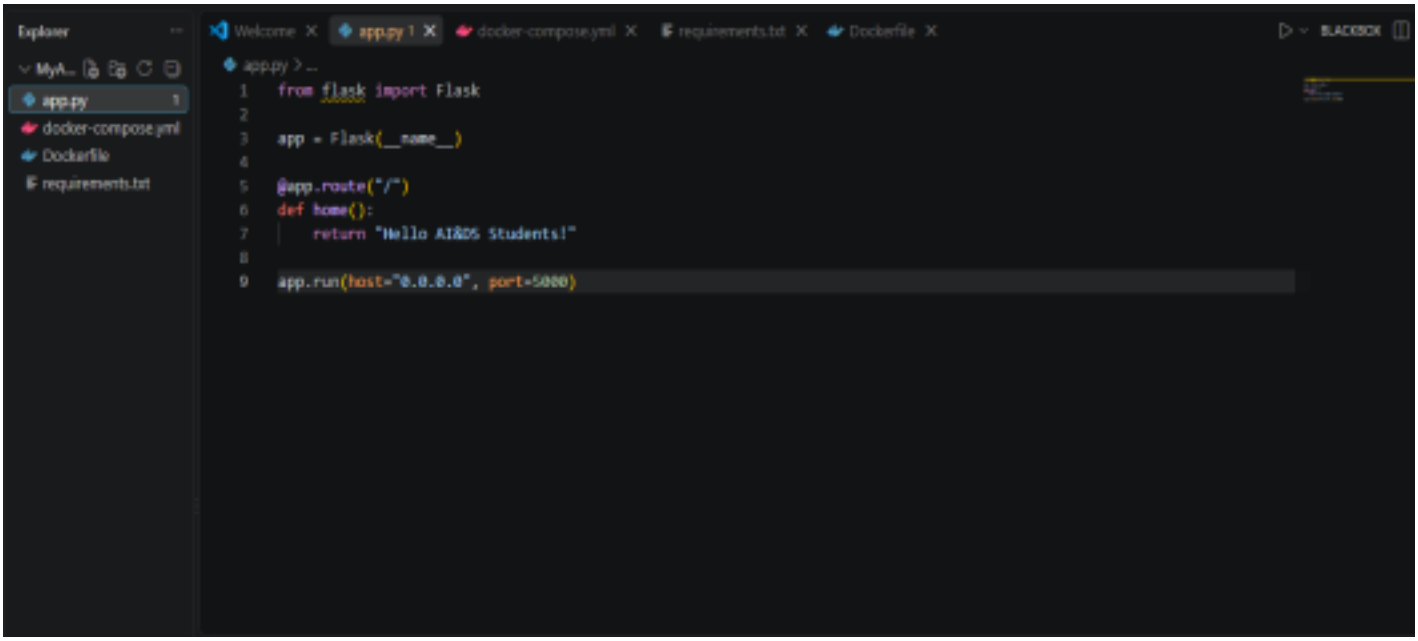


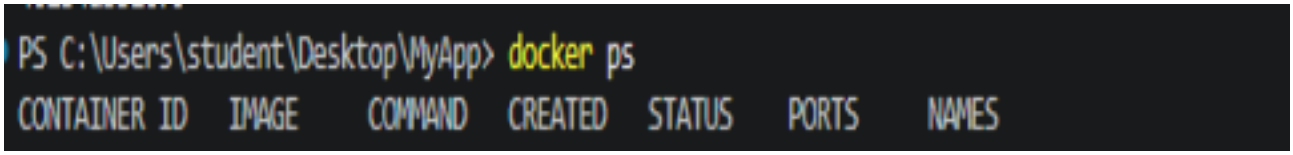
PART B
(PART B: TO BE COMPLETED BY STUDENTS)

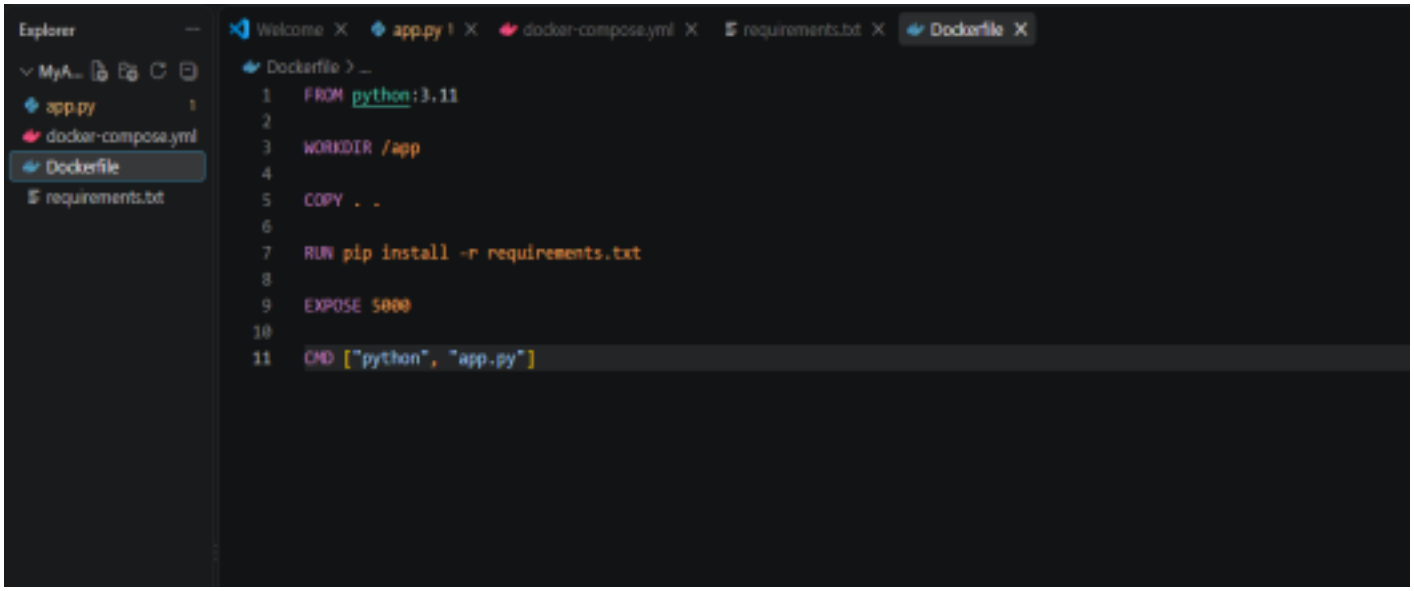
Roll No. A27	Name: Prajwal Ashok Zolage
Class: BE–AI&DS	Batch: A1
Date of Experiment:21/09/2026	Date of Submission: 21/09/2026

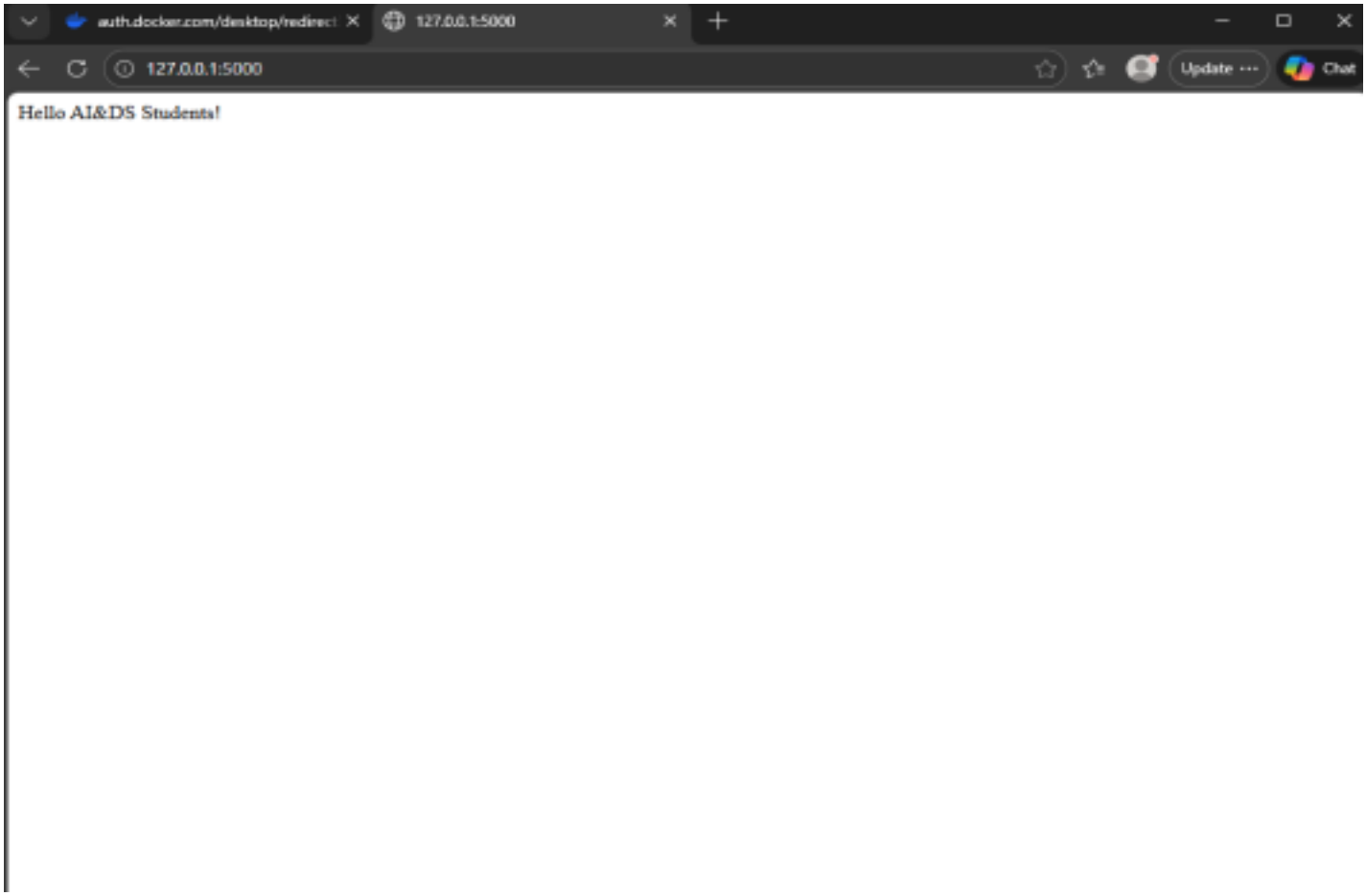
B.1 Code Screenshot:



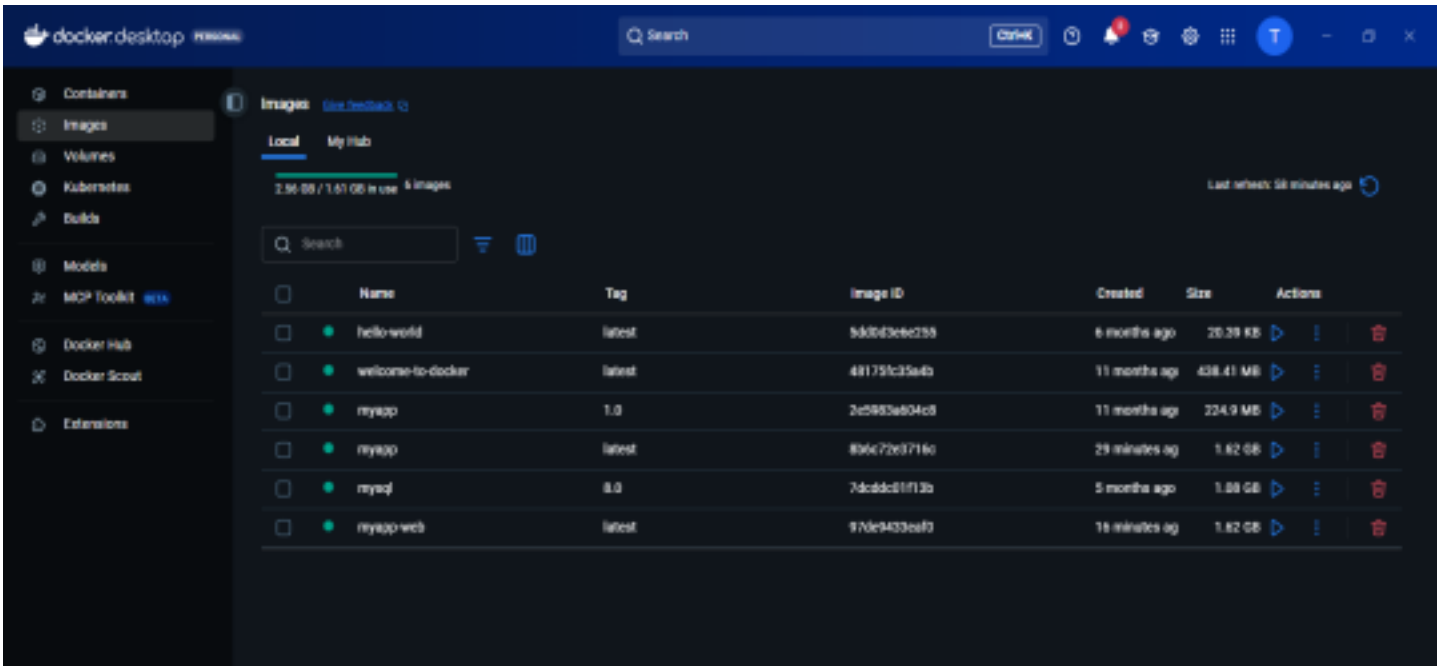


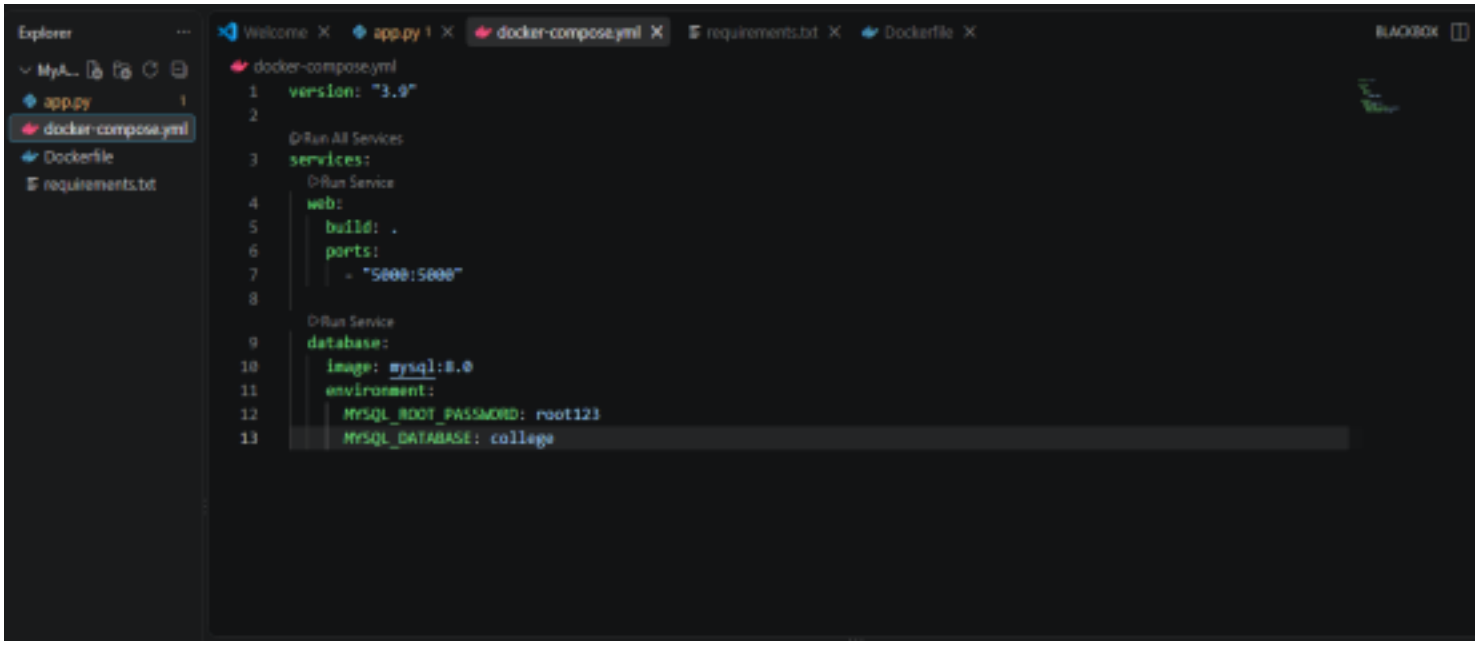



Explorer
MyA...
app.py
docker-compose.yml
Dockerfile
requirements.txt
Welcome
app.py
docker-compose.yml
requirements.txt
Dockerfile
Dockerfile > ...
1 FROM python:3.11
2
3 WORKDIR /app
4
5 COPY . .
6
7 RUN pip install -r requirements.txt
8
9 EXPOSE 5000
10
11 CMD ["python", "app.py"]


auth.docker.com/desktop/redirect
127.0.0.1:5000
127.0.0.1:5000
Update
Chat
Hello AI&DS Students!

```
PS C:\Users\student\Desktop\MyApp> docker compose up -d
time="2026-09-21T12:23:19+05:30" level=warning msg="C:\\Users\\student\\Desktop\\MyApp\\docker-compose.yml: the attribute `version` is obsolete, it will
 be ignored, please remove it to avoid potential confusion"
[+] Running 12/12
 ✓ database Pulled                                                                                                                                  45.9s
[+] Building 7.7s (12/12) FINISHED
 => [internal] load local bake definitions                                                                                                           0.0s
 => => reading from stdin 506B                                                                                                                       0.0s
 => [internal] load build definition from Dockerfile                                                                                                 0.0s
 => => transferring dockerfile: 163B                                                                                                                 0.0s
 => [internal] load metadata for docker.io/library/python:3.11                                                                                       1.7s
 => [auth] library/python:pull token for registry-1.docker.io                                                                                        0.0s
 => [internal] load .dockerignore                                                                                                                    0.0s
 => => transferring context: 2B                                                                                                                      0.0s
 => [1/4] FROM docker.io/library/python:3.11@sha256:70e8e937c1da72df1688e883a2108b0feff03381187fa78eca6d9b10df411320                                 0.0s
 => => resolve docker.io/library/python:3.11@sha256:70e8e937c1da72df1688e883a2108b0feff03381187fa78eca6d9b10df411320                                 0.0s
 => [internal] load build context                                                                                                                    0.0s
 => => transferring context: 345B                                                                                                                    0.0s
 => CACHED [2/4] WORKDIR /app                                                                                                                        0.0s
 => [3/4] COPY . .                                                                                                                                   0.2s
 => [4/4] RUN pip install -r requirements.txt                                                                                                        4.1s
 => exporting to image                                                                                                                               1.3s
 => => exporting layers                                                                                                                              0.9s
 => => exporting manifest sha256:799af5667e183076da9b8d17f2ad3148a1fad225461562f80bbf5464593ea5fc                                                    0.0s
 => => exporting config sha256:82a897ceab4b2cc191cb390c961dbb929b2a7bf1096ec6958afc47f1e4c3476c                                                      0.0s
 => => exporting attestation manifest sha256:e43253d6387c12dd6abc0ffc56763512a4a8b532dc279b817b1a6868a092936f                                        0.0s
 => => exporting manifest list sha256:97de9433eaf05d59388709da3846e5d596617e036bef9aab4009d4ee19292ed                                               0.0s
 => => naming to docker.io/library/myapp-web:latest                                                                                                  0.0s
 => => unpacking to docker.io/library/myapp-web:latest                                                                                               0.3s
 => resolving provenance for metadata file                                                                                                           0.0s
[+] Running 4/4
 ✓ myapp-web                 Built                                                                                                                   0.0s
 ✓ Network myapp_default     Created                                                                                                                 0.1s
 ✓ Container myapp-database-1 Started                                                                                                                0.5s
 ✓ Container myapp-web-1     Started                                                                                                                 0.5s
```





B.2 Input and Output:

1) Docker version

PS C:\Users\student\Desktop\MyApp> docker --version
Docker version 28.4.0, build d8eb465

2) Docker image

PS C:\Users\student\Desktop\MyApp> docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
myapp-web latest 97de9433eaf0 5 minutes ago 1.62GB
myapp latest 8b6c72e3716c 19 minutes ago 1.62GB
mysql 8.0 7dcddc01f13b 4 months ago 1.09GB
hello-world latest 5dd0d3e6e255 6 months ago 20.4kB
welcome-to-docker latest 48175fc35a4b 10 months ago 438MB
myapp 1.0 2e5983a604c8 10 months ago 225MB

3) Docker container

PS C:\Users\student\Desktop\MyApp> docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
da4eee06e393 myapp-web "python app.py" 6 minutes ago Up 6 minutes 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp myapp-web-1
b1c1945693c5 mysql:8.0 "docker-entrypoint.s…" 6 minutes ago Up 6 minutes 3306/tcp, 33060/tcp myapp-database-1

4) Browser output

http://localhost:5000

5) Docker Compose

PS C:\Users\student\Desktop\MyApp> docker compose up -d
time="2026-09-21T12:23:19+05:30" level=warning msg="C:\\Users\\student\\Desktop\\MyApp\\docker compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 12/12
✔ database Pulled 45.9s [+] Building 7.7s (12/12) FINISHED
=> [internal] load local bake definitions 0.0s => => reading from stdin 506B 0.0s => [internal] load build definition from Dockerfile 0.0s => => transferring dockerfile: 163B 0.0s => [internal] load metadata for docker.io/library/python:3.11 1.7s
=> [auth] library/python:pull token for registry-1.docker.io 0.0s => [internal] load .dockerignore 0.0s =>

=> transferring context: 2B 0.0s => [1/4] FROM docker.io/library/python:3.11@sha256:70e8e937c1da72df1688e883a2108b0feff03381187fa78eca6d9b10df411320 0.0s => => resolve docker.io/library/python:3.11@sha256:70e8e937c1da72df1688e883a2108b0feff03381187fa78eca6d9b10df411320 0.0s => [internal] load build context 0.0s => => transferring context: 345B 0.0s => CACHED [2/4] WORKDIR /app 0.0s => [3/4] COPY . . 0.2s => [4/4] RUN pip install -r requirements.txt 4.1s => exporting to image 1.3s => => exporting layers 0.9s => => exporting manifest sha256:799af5667e183076da9b8d17f2ad3148a1fad225461562f00bbf5464593ea5fc 0.0s => => exporting config sha256:82a897ceab4b2cc191cb390c961dbb929b2a7bf1096ec6958afc47f1e4c3476c 0.0s => => exporting attestation manifest sha256:e43253d6387c12dd6abc6ffc56763512a4a8b532dc279b817b1a6868a692936f 0.0s => => exporting manifest list sha256:97de9433eaf05d593807049da3846e5d596617e036bef9aab4009d4ee19292ed 0.0s => => naming to docker.io/library/myapp-web:latest 0.0s => => unpacking to docker.io/library/myapp-web:latest 0.3s => resolving provenance for metadata file 0.0s [+] Running 4/4 ✔ myapp-web Built 0.0s ✔Network myapp_default Created 0.1s ✔Container myapp-database-1 Started 0.5s ✔ Container myapp-web-1 Started

B.3 Conclusion:

In this experiment, we successfully learned the basics of Docker containerization. We created a Flask application, prepared a Dockerfile, built a Docker image, and ran the application inside a Docker container. We also created a Docker Compose file and successfully deployed the Flask web application and MySQL database as multiple services. Thus, we understood how Docker and Docker Compose can be used to package, run, and manage applications and their dependencies.

Viva Questions

Docker vs Docker Compose vs Kubernetes

Feature	Docker	Docker Compose	Kubernetes
Purpose	Create container	Multiple containers	Large-scale orchestration
Config file	Dockerfile	compose.yml	deployment.yml
Scaling	Manual	Limited	Automatic

Best for	One app	Small projects	Production systems

Q1. What is Docker? A platform for containerizing applications.

Answer: Docker is a platform for containerizing applications.

Q2. What is Dockerfile? A text file containing instructions to build a Docker image.

Answer: A Dockerfile is a text file containing instructions used to build a Docker image.

Q3. Difference between Image and Container?

Image	**Container**
Template	Running instance
Static	Dynamic

Q4. What is Docker Compose? A tool for defining and running multiple containers using one YAML file.

Answer: Docker Compose is a tool for defining and running multiple containers using one YAML file. Q5. Why use Docker Compose? It starts web server, database, cache, and other services with a single command.

Answer: It allows services such as a web server, database, and cache to be started and managed using a single configuration and command.